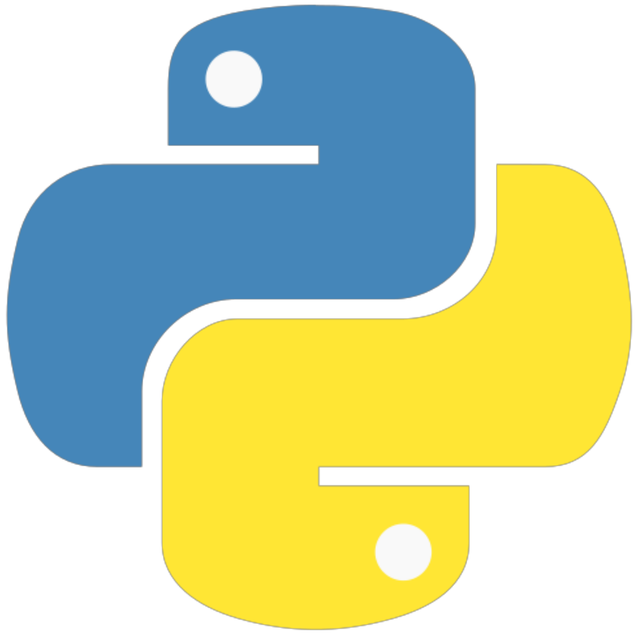
UNIVERSITÄT OSNABRÜCK

Programmieren für PsychologInnen

EEG-Daten in Python



Allgemeine Psychologie I
B.Sc. Psychologie / M.Sc. Psychologie - Seminar Programmieren
11.07.2025

Merle Sagehorn, M.Sc.

Today

- Vor der Durchführung
  - Triggerlisten
  - Programm schreiben: Python vs. Unity

- Vor der Datenanalyse
  - Preprocessing von EEG-Daten

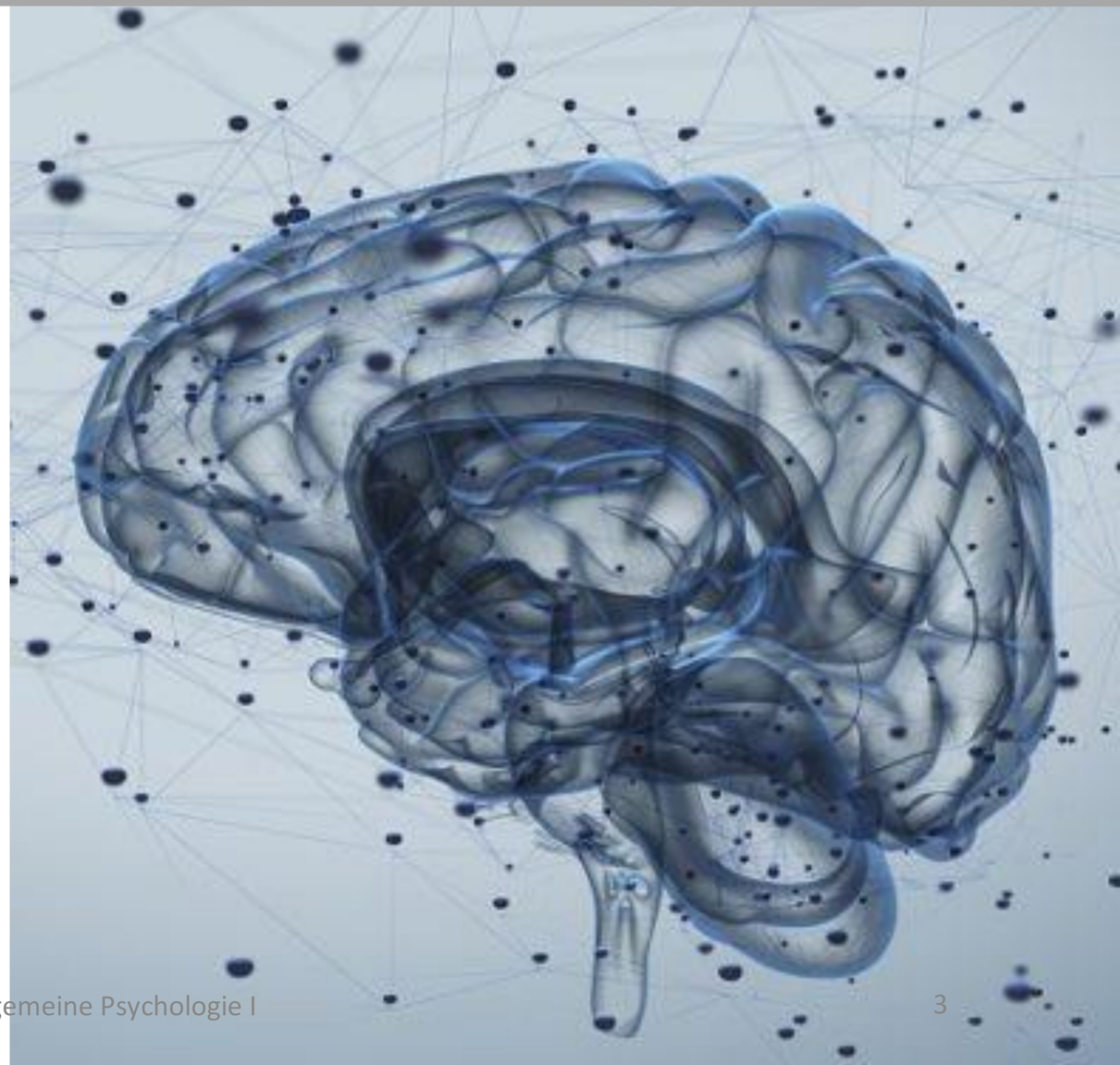
- Datenanalyse & Visualisierung
  - MNE

EEG-Experiment: Repetition Suppression

- = Korrelat erfolgreicher Enkodierung

- RS-Paradigma: zweifache Präsentation von Objekten (oder Tönen o.ä.)
  - priming-bedingte Abnahme neuronaler Aktivität bei der zweiten Präsentation eines Stimulus
  - sichtbar in späteren EKP-Komplexen (nach P1 und N1)

- RS-Effekt als Prädiktor für Gedächtnisleistung



EEG-Experiment: Repetition Suppression

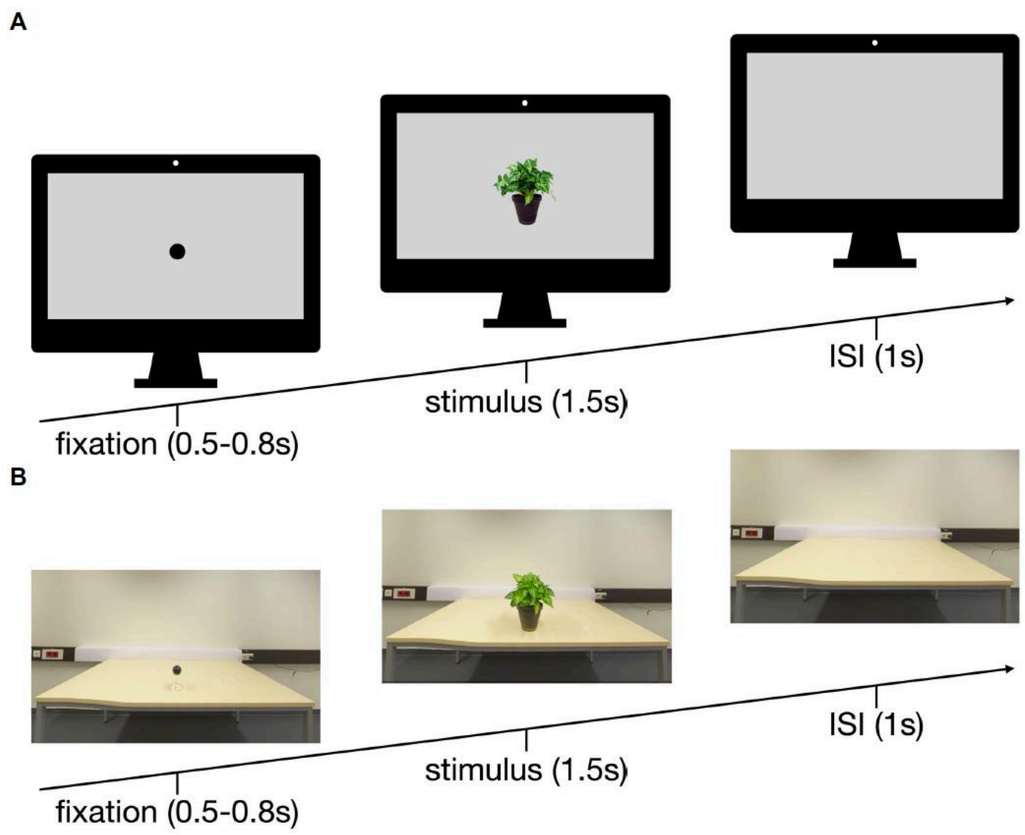


FIGURE 1
Setup of the trials in the PC **(A)** and the VR **(B)** conditions: 0.5–0.8s fixation, 1.5s stimulus, 1s inter stimulus interval (ISI).

Design:

- 1 Präsentation: 60 Objekte
- 2 Präsentationen: 60 Objekte

- Wiederholung mit 1, 2 oder 3 anderen Objekten zwischen der ersten und zweiten Präsentation

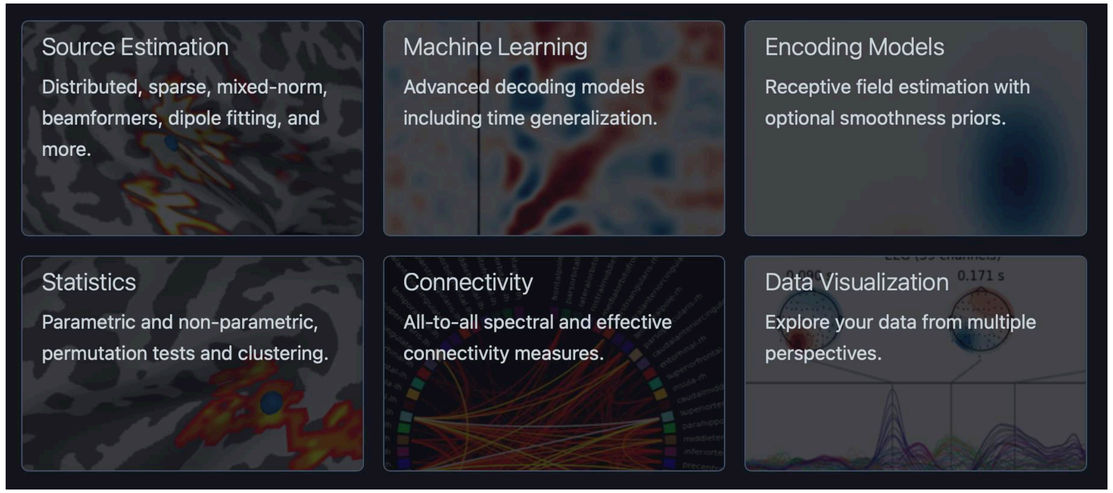
- Objekte entweder am PC oder in VR

Johnsdorf, M., Kisker, J., Gruber, T., & Schöne, B. (2023). Comparing encoding mechanisms in realistic virtual reality and conventional 2D laboratory settings: Event-related potentials in a repetition suppression paradigm. *Frontiers in Psychology*, *14*, 1051938.

MNE

- Open-Source-Python-Paket zum Erforschen, Visualisieren und Analysieren menschlicher neurophysiologischer Daten: MEG, EEG, sEEG, ECoG, NIRS, etc.

- unabhängige Komponentenanalyse (ICA), Zeit-/ Frequenzanalyse, Artefaktbereinigung, ereigniskorrelierte Statistiken und mehreren nützlichen Visualisierungsmodi für gemittelte und Einzel-Trial-Daten

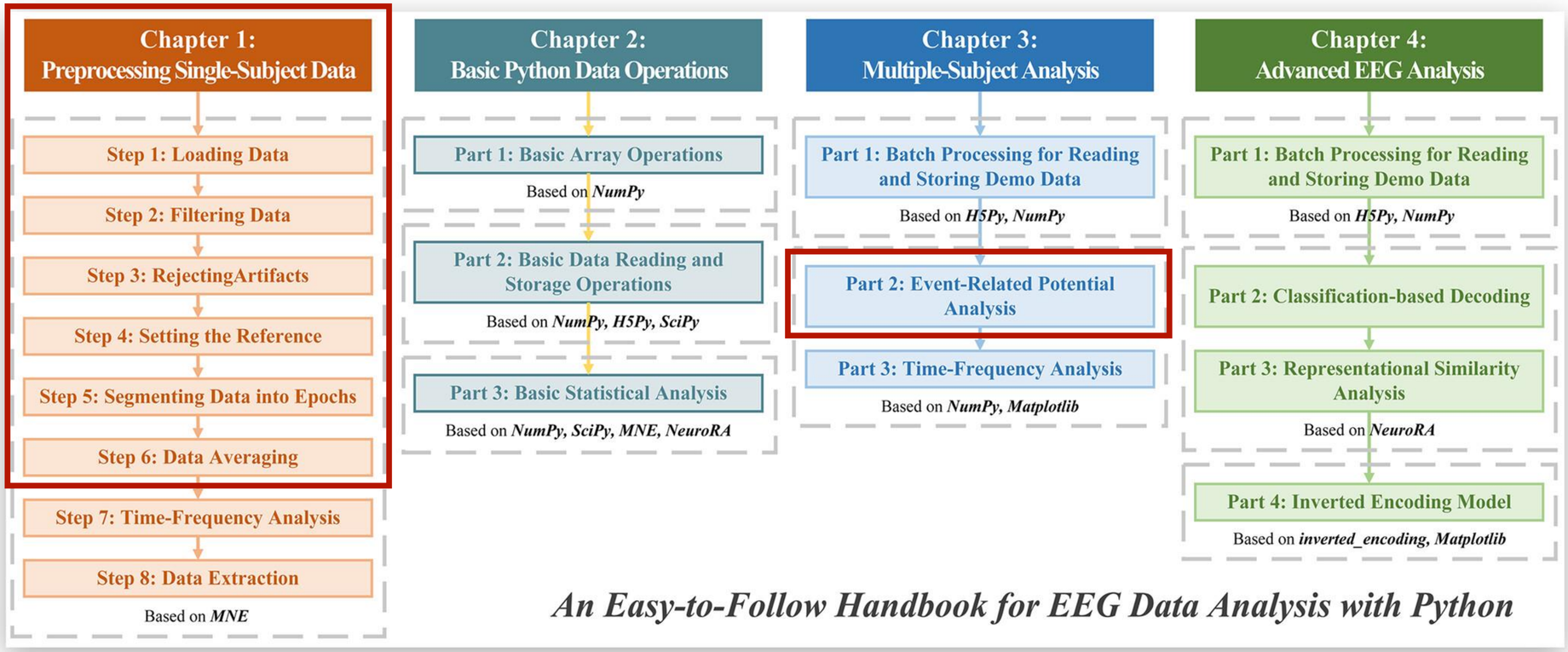


Gramfort, A., Luessi, M., Larson, E., Engemann, D.A., Strohmeier, D., Brodbeck, C., Goj, R., Jas, M., Brooks, T., Parkkonen, L., & Hämäläinen, M.S. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience, 7*(267), 1–13. doi:10.3389/fnins.2013.00267.

MNE

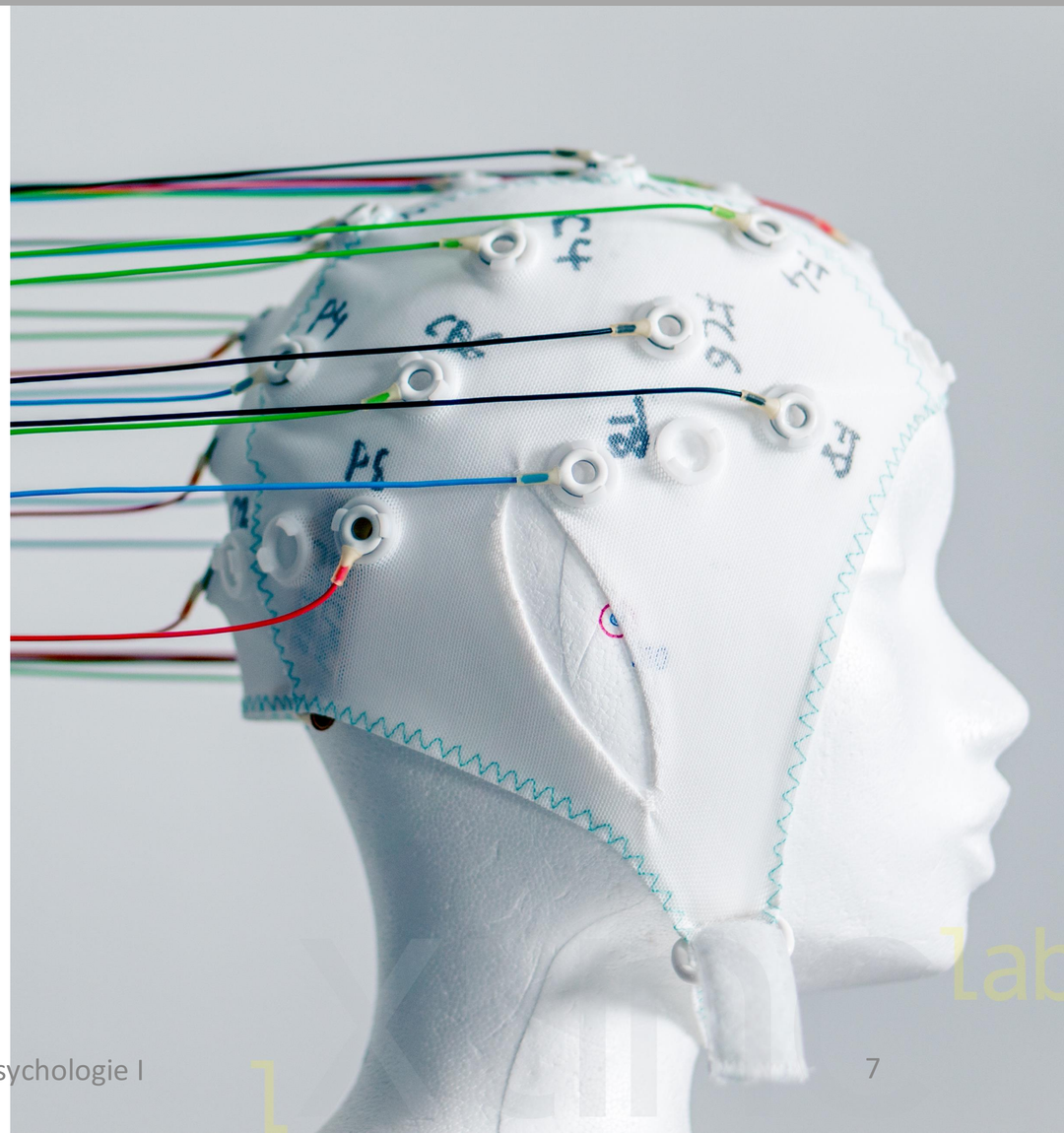
An easy-to-follow handbook for electroencephalogram data analysis with Python

Zitong Lu[1] | Wanru Li[2] | Lu Nie[3] | Kuangshi Zhao[4]



EEG-Preprocessing

- Re-Referenzierung
- Filtern
- Datenbereininung
- Interpolation
- Linear Detrending
- ICA zur Artefaktbereinigung
- Epochierung
- Baseline Korrektur
- Grand Mean



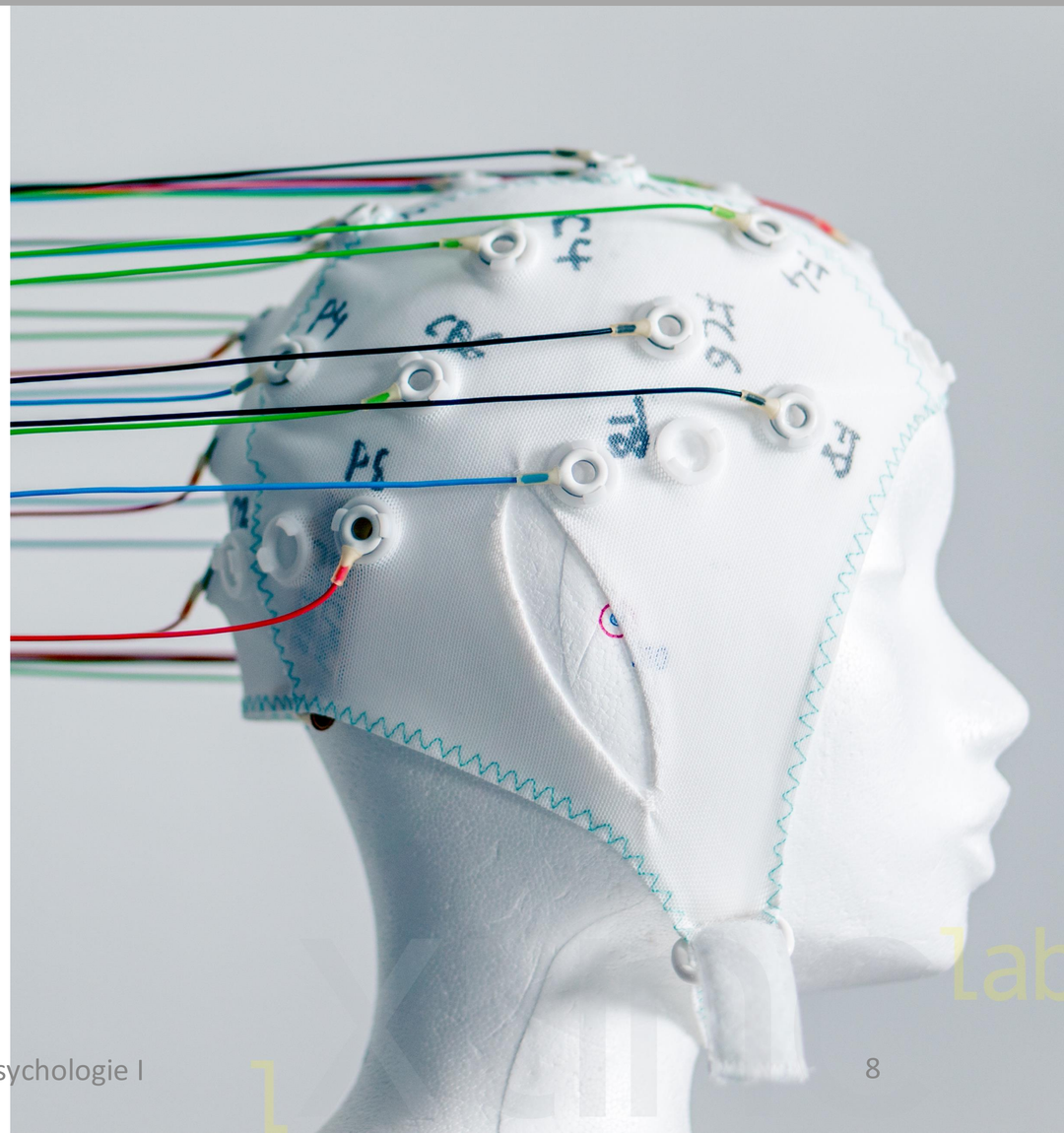
EEG-Preprocessing

- Daten Laden

```
# EEG Daten einlesen
raw = read_raw_eeglab('EEG_DATA_FILE.set', preload = True)
```

- Re-Referenzierung*

```
# Rereferenzierung
raw_avg_ref = raw.set_eeg_reference(ref_channels = "average")
```



Exkurs: Re-Referenzierung

- EEG-Signal = Unterschiede in den elektrischen Potenzialen zwischen zwei Punkten —> Spannungen in Mikrovolt (µV)
- Signal an einem Kanal = die Differenz des elektrischen Potenzials zu einer anderen Aufzeichnungsstelle (Ground)

- **Ground-Elektrode** = mit Ground-Kreis des Verstärkers verbundene Elektrode
  - empfänglich für Störsignale
  - —> ground-bedingtes Rauschen in den Daten
- **Online-Referenz** = Baseline-Channel für alle anderen EEG-Channels
  - ([channel – ground] – [online reference – ground] = channel – online reference)
  - —> Signal der EEG-Channels als Potentialdifferenz zur Online-Referenz

- **Re-Referenzierung** = EEG-Channels in Relation zu anderer Referenz (i.d.R. Cz oder Fz oder average, manchmal auch Ohr/Wange)

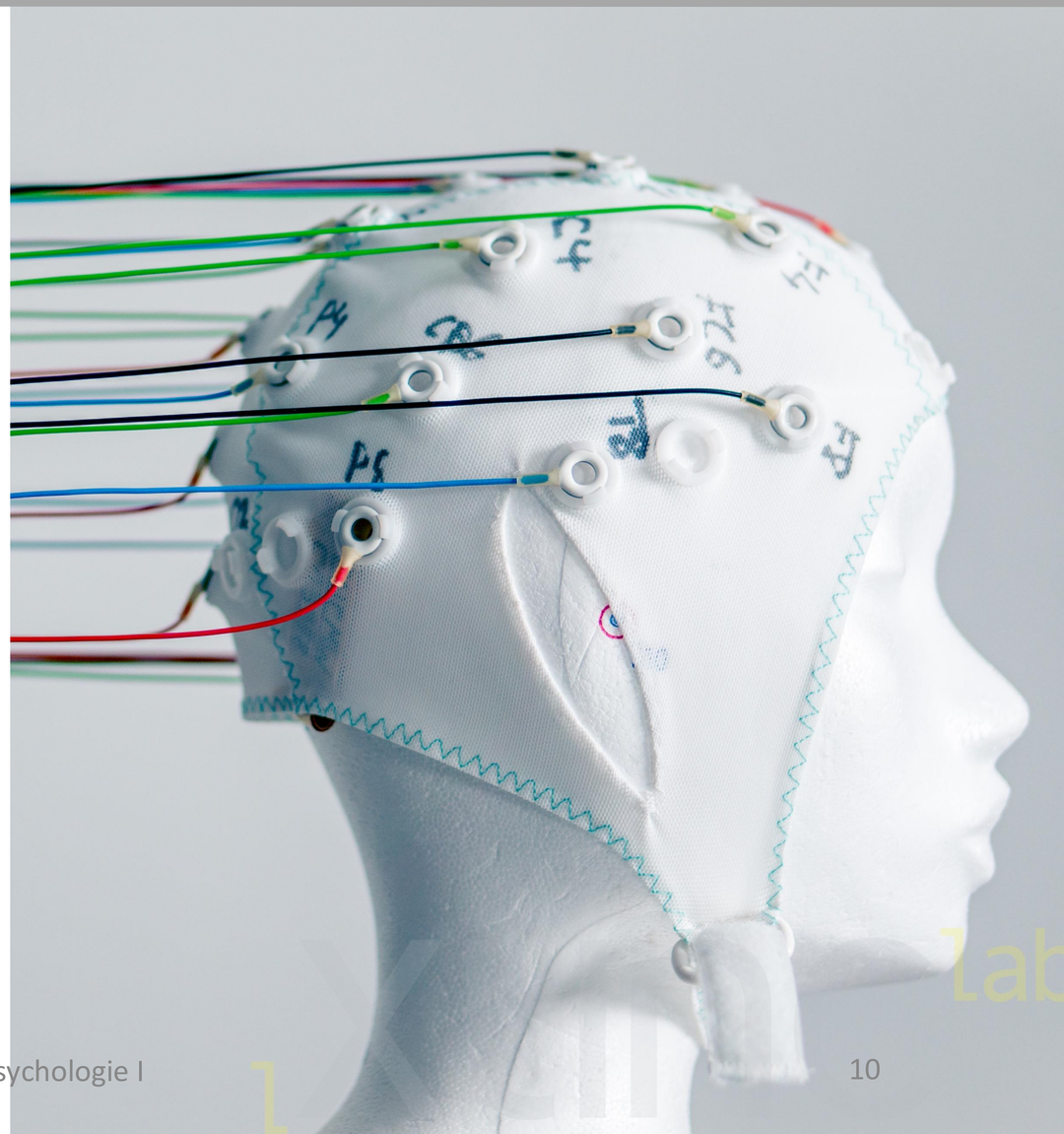
EEG-Preprocessing

- Filtern: High-pass, Low-pass, … , aufpassen weil leicht 0 Hz Frequenzen
  ↳ normal 2,5 - 30 Hz

- Datenbereininung: Ausreißer-Kanäle identifizieren
  - Local Outlier Factor (LOF) → Kanäle als Ausreißer mit wesentlich geringerer Dichte als ihre Nachbarn identifizieren
  - Kanäle als Ausreißer mit deutlich stärkerem Signal als der Durchschnitt (z.B. 2 Standardabweichungen)

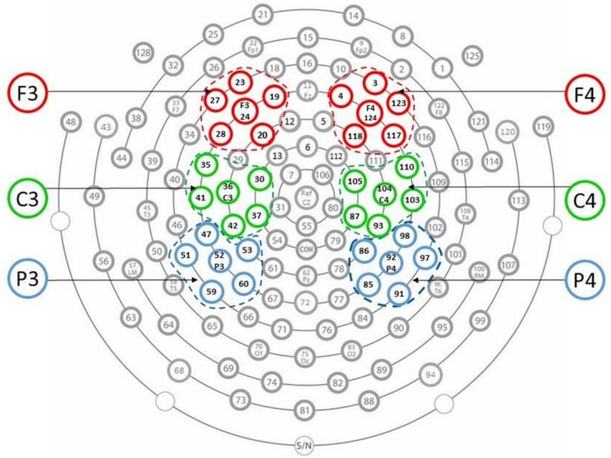
- Interpolation: „bad“ channels interpolieren, d.h. Mittelwert der umliegenden Kanäle berechnen und damit den Kanal ersetzen*

- Linear Detrending: langsame „drifts“ in den Daten entfernen (per Channel)*

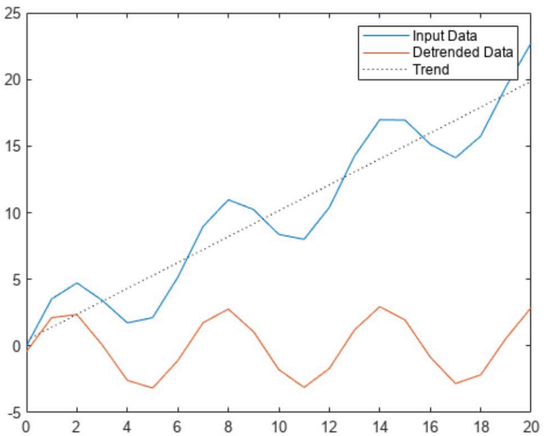


Exkurs: Interpolation & Linear Detrending

- Interpolation = „unechtes“ EEG-Signal auf der Grundlage real existierenden Signals erzeugen
  - Idee: räumliche Verteilung der Elektroden auf der Kopfoberfläche —> Nachbar-Elektroden zeichnen sehr ähnliches Signal auf —> schlechtes Signal durch Mittelwert der Nachbar-Elektroden ersetzen



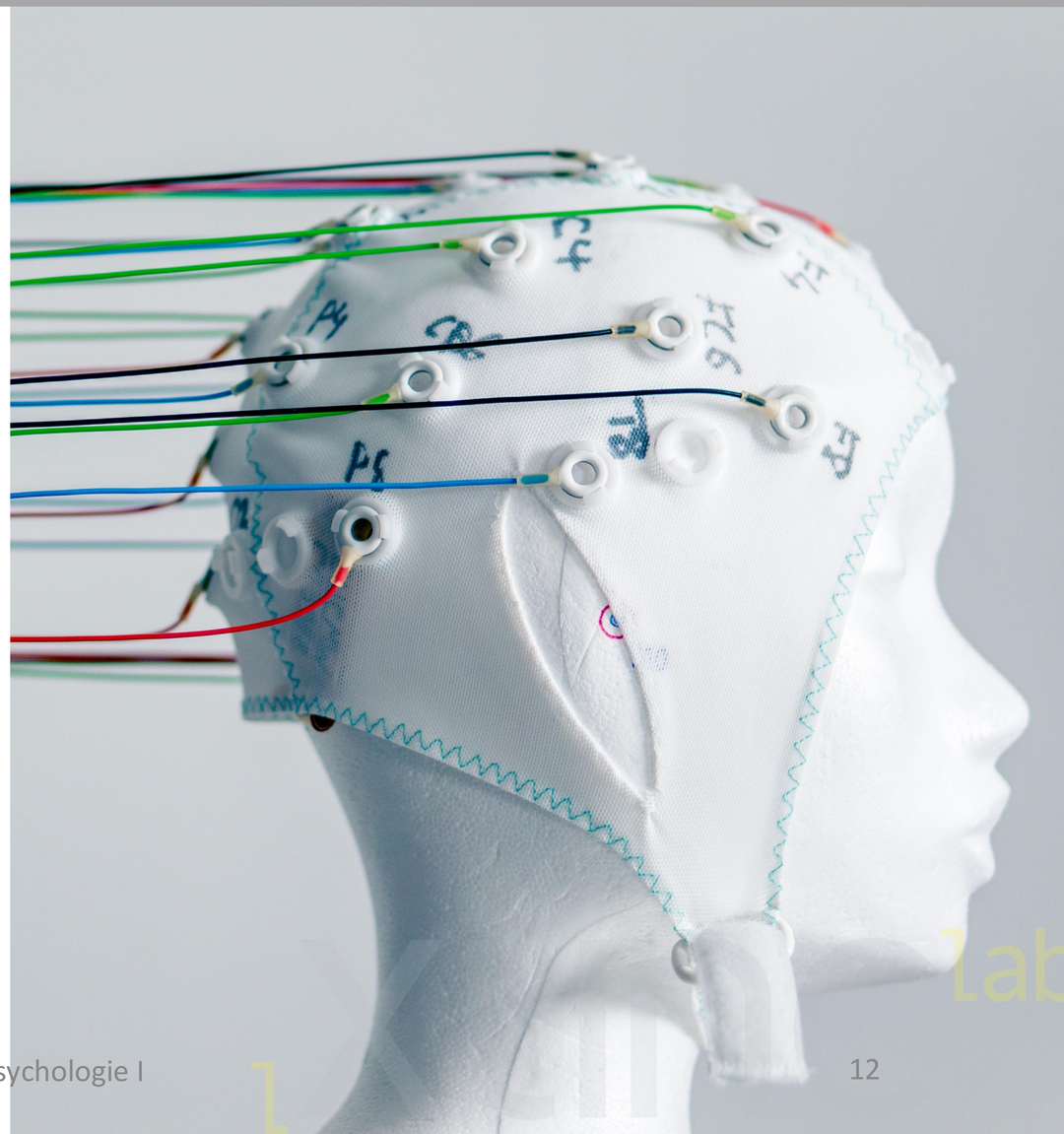
- Lineares Detrending entfernt langsame Drifts aus den Daten
  - ≈ langsames „Wandern“ des Signals



- signal.detrend () —> entfernt die beste Anpassungsgerade (Trend) aus den Daten und gibt die übrigen Daten zurück

EEG-Preprocessing

```python
# Bandpassfilter zwischen 0.25 und 30 Hz
raw.filter(0.25, 30., fir_design='firwin')

# Schlechte Kanäle identifizieren
raw.find_bad_channels_lof(raw, n_neighbors=20, picks=None,
                          metric='euclidean', threshold=1.5)

# Interpolation
raw.interpolate_bads(reset_bads = True)

# Linear Detrending
data = raw.get_data()
for i in range(data.shape[0]):  # Für jeden Kanal
    data[i, :] = signal.detrend(data[i, :], type='linear')
raw._data = data
```

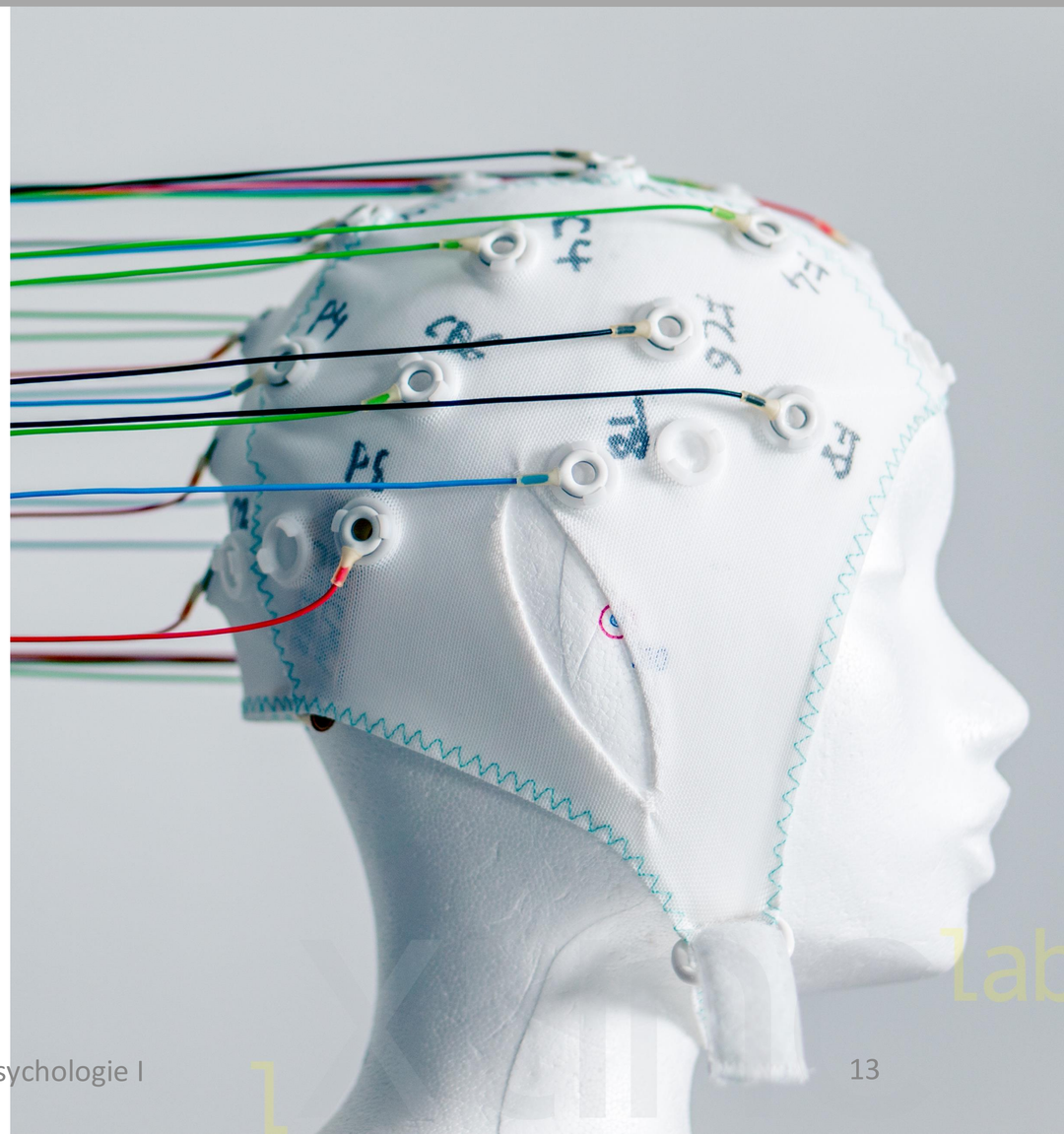


EEG-Preprocessing

- ICA zur Artefaktbereinigung:

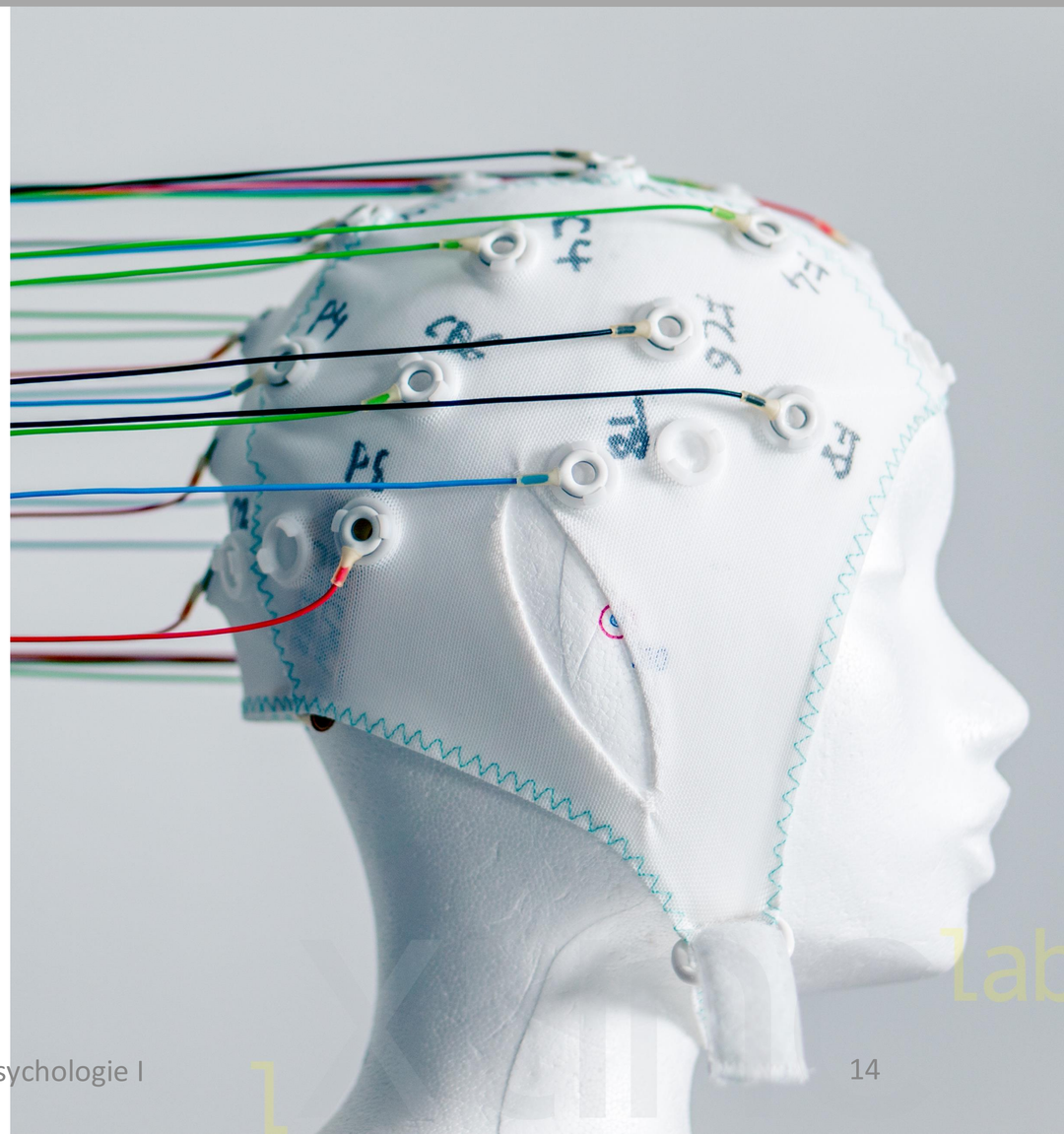
Entfernen/Subtrahieren von in die Daten eingebetteten Artefakten (Muskeln, Augenblinzeln oder Augenbewegungen), ohne die betroffenen Datenbereiche zu entfernen

- ICA-Komponenten berechnen
- Komponenten anzeigen lassen
- Komponenten auswählen und entfernen
- → In Matlab auch automatisiert möglich mithilfe von Schwellenwerten & Abgleich mit Datenbank an typischen Artefakten (ICLabel).

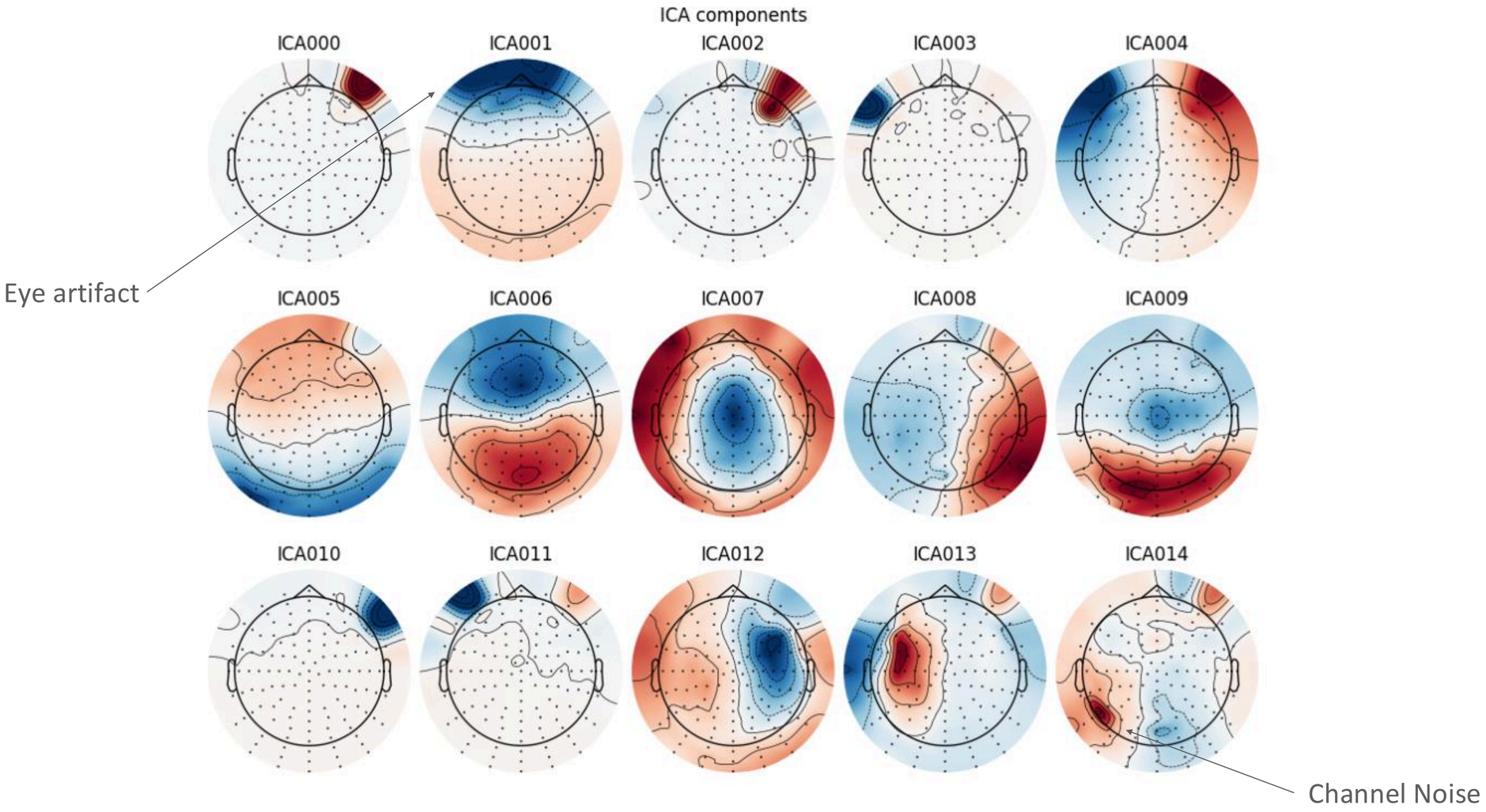


EEG-Preprocessing

```python
# ICA fitting
ica = ICA(n_components=15, max_iter="auto")
ica.fit(raw)
ica

# ICA visualization
ica.plot_sources(raw, show_scrollbars=False)
ica.plot_components()


# ICA component removal
ica.exclude = [::]  # alle auswählen
ica.apply(raw)
```



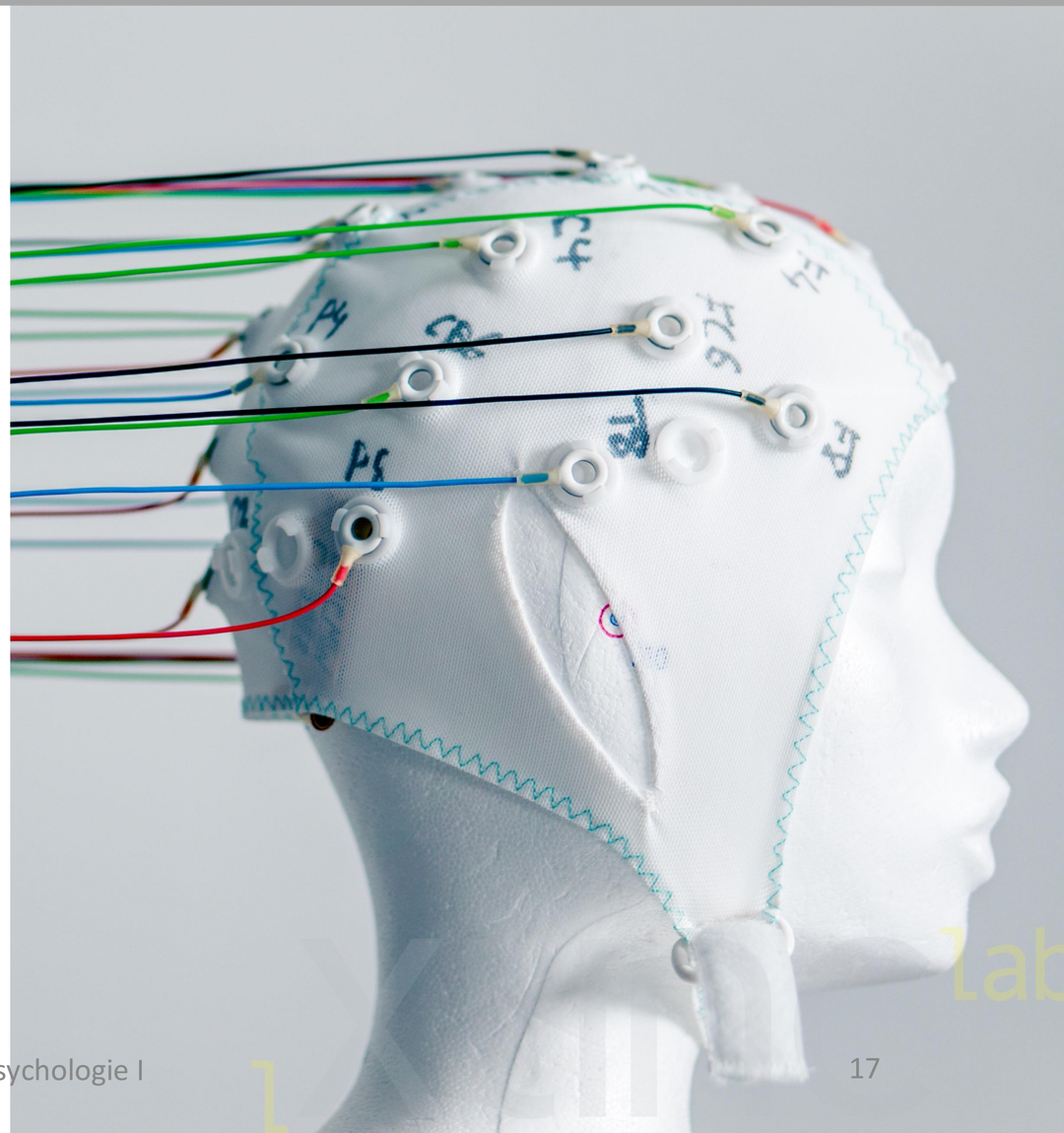
EEG-Preprocessing



EEG-Preprocessing

- Epochierung: Zeitfenster um Event herum ausschneiden
  = oft nötig
  => [illegible]
- Event = i.d.R. irgendeine Form von Stimuluspräsentation
  = Trigger als Marke

```
# Ereignisse finden
events = mne.find_events(raw, stim_channel='Trigger_channel')

# Epochen erstellen
event_id = {'TriggerOn': 1, 'TriggerOff': 2}  # Angepasste Werte und Event-IDs
epochs = mne.Epochs(raw, # Daten
                    events, # extrahierte Events
                    event_id,
                    tmin = -0.5, # Start der Epoche relativ zum Event
                    tmax = 1.5, # Ende der Epoche relativ zum Event
                    baseline = (None, 0),
                    preload = True)
```



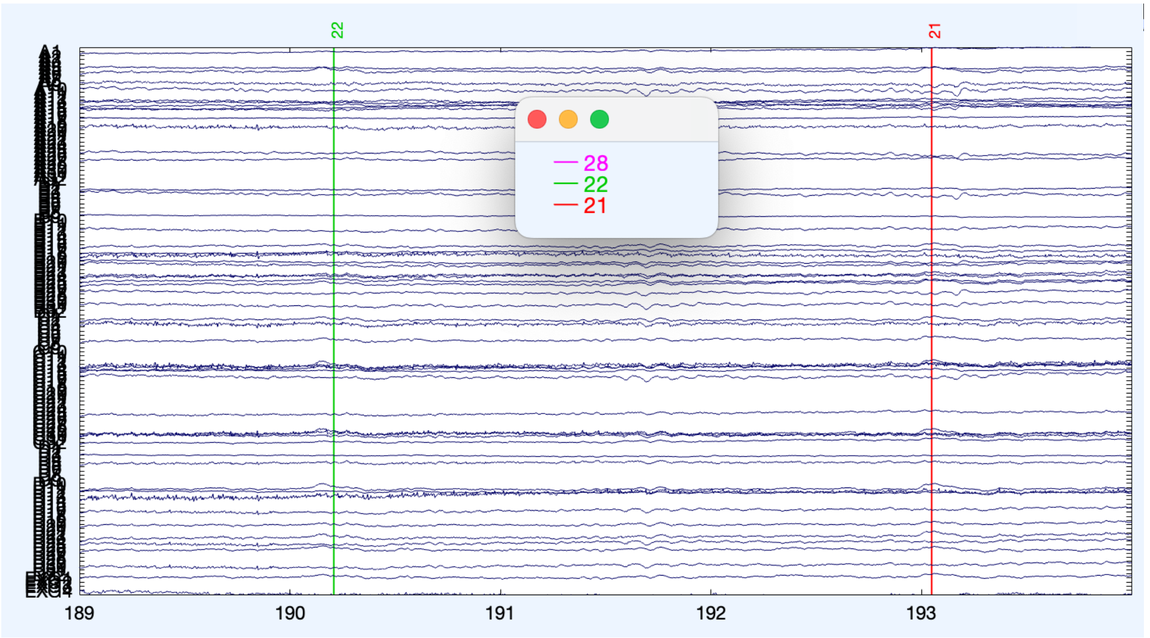
Trigger im EEG

- Trigger = Zeitstempel in den kontinuierlichen Daten

  = ist auch Ortsd des Stimulus -> sehen als Start des Prozess
  ↳ liefert präzises Zeitfenster

—> markieren den Zeitpunkt bestimmter Events im EEG-Datenstrom

—> dienen als Marker, um später ein genaues Zeitfenster um das Event of interest herum ausschneiden zu können (z.B. für EKP-Analyse)

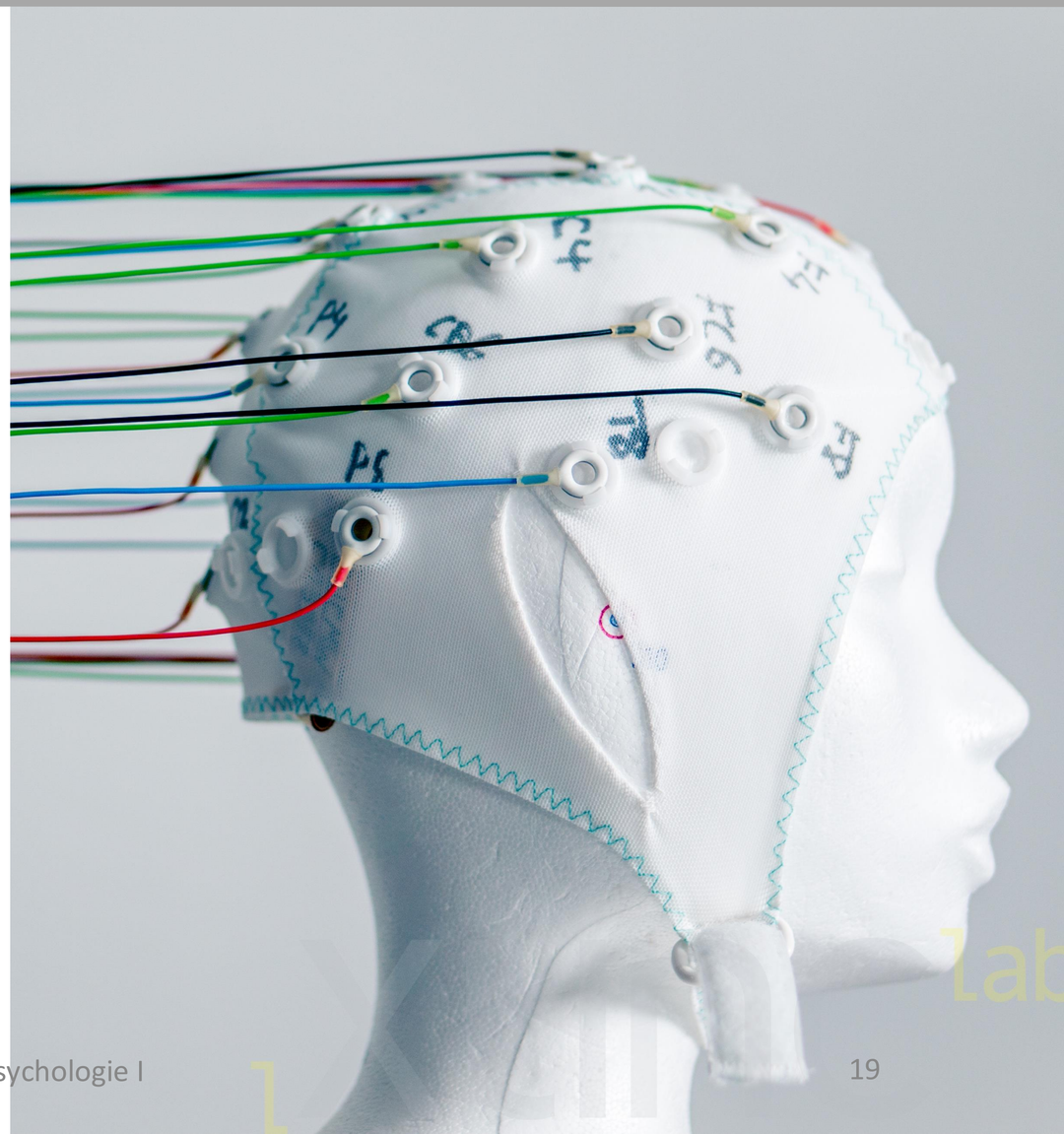


21 —> Erstpräsentation 2D Stimulus
22 —> Zweitpräsentation 2D Stimulus
28 —> nicht wiederholter 2D Stimulus

EEG-Preprocessing

- Baseline Korrektur: Verwendung der EEG-Aktivität vor dem Auftreten eines externen Ereignisses, zur Korrektur der Aktivität in einem Post-Stimulus-Intervall

```
# Baseline_Korrektur
baseline_tmin, baseline_tmax = -300, 0
baseline = (baseline_tmin, baseline_tmax)
trad_aud = epochs.average().apply_baseline(baseline)
```



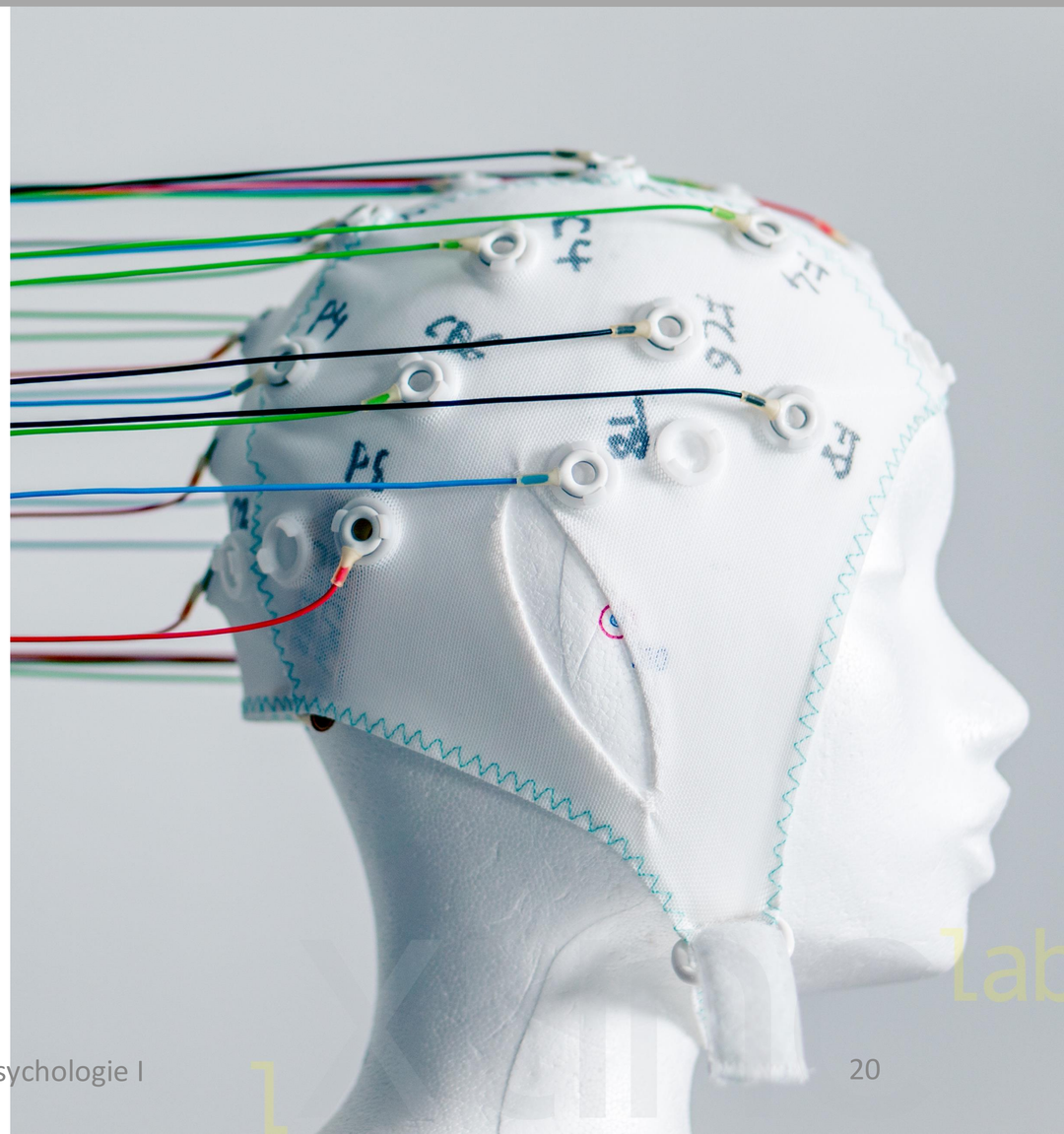
EEG-Preprocessing

- Grand mean: z.B. über alle Trials einer VP

```
# Grand Mean
grand_average = mne.grand_average(epochs.average())
```

- Daten speichern

```
# Daten speichern
save_path = 'SAVE_DATA_FILE.fif'
raw.save(save_path, overwrite = True)
```



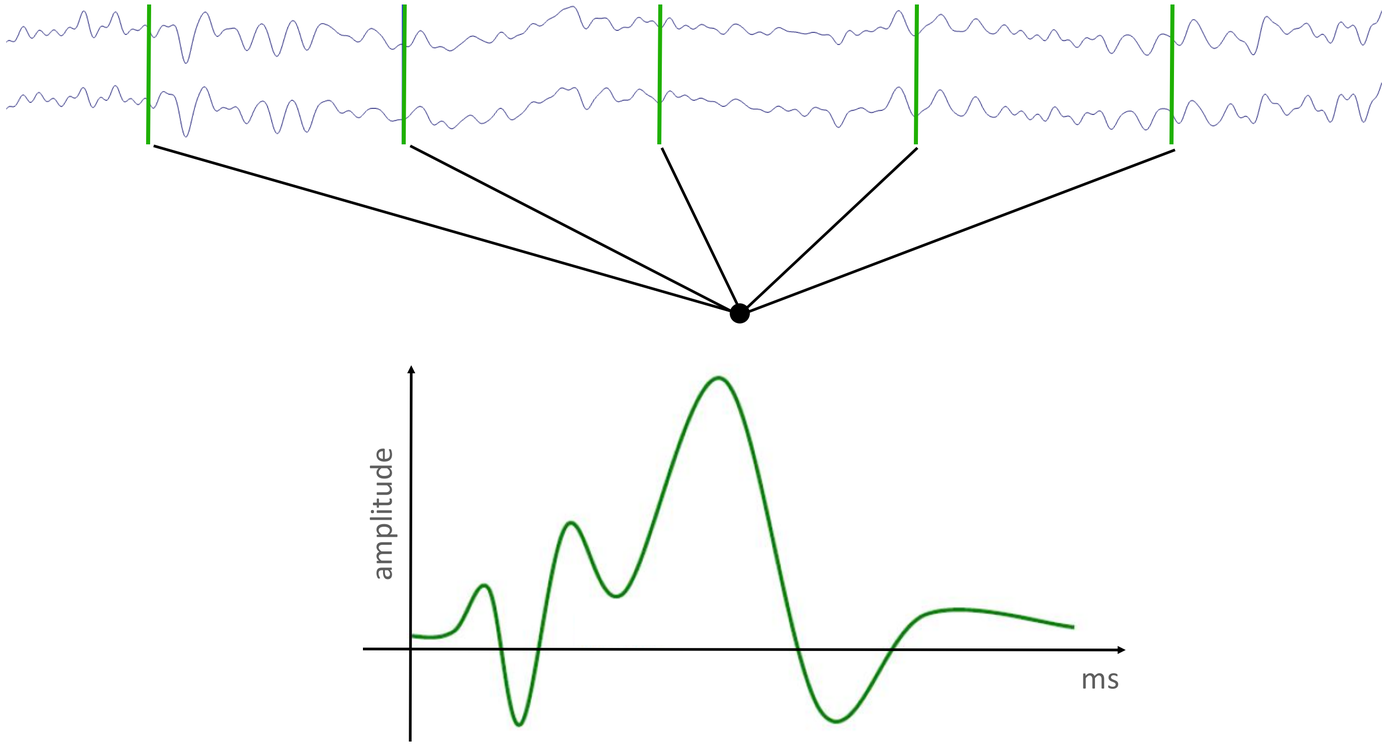
Grand Mean | EKP

- EKP —> Mittelwert des EEG-Signal für ein festgelegtes (event-gelocktes) Zeitfenster…

  …über einen/mehrere/viele **Kanäle**

  …viele **Trials**

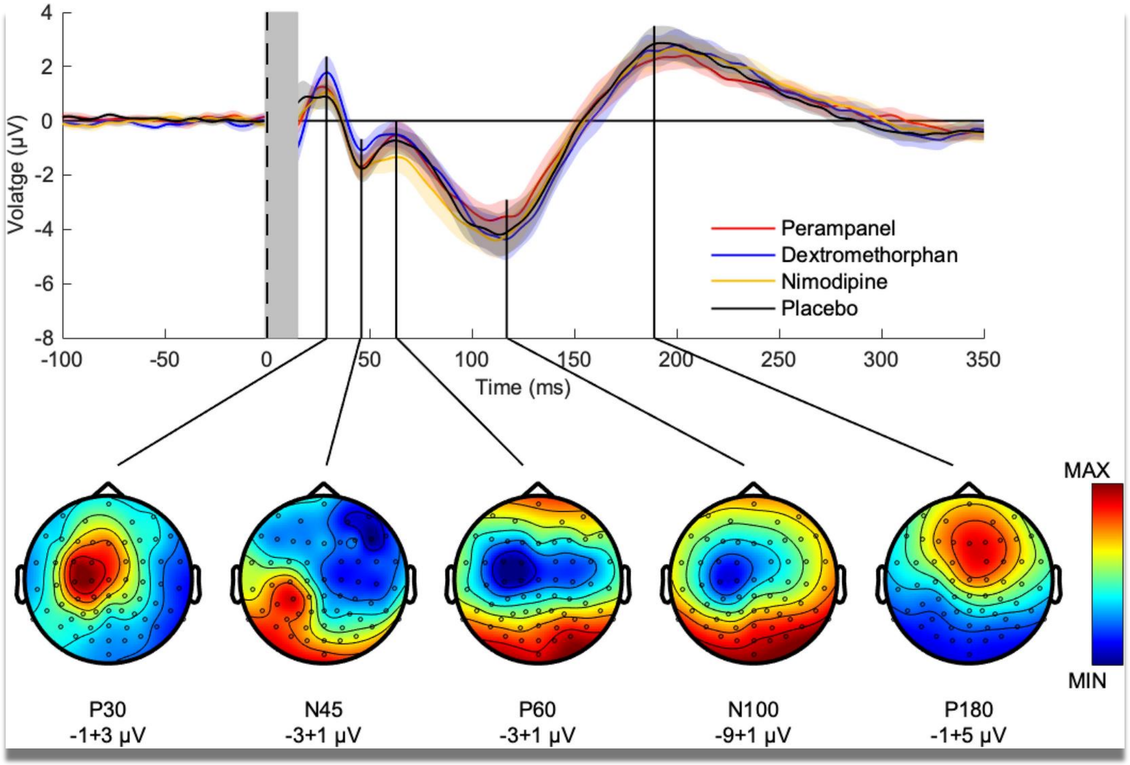
  …viele **VPs**



Visualisierung von EEG Daten

- Lineplots
  - Plot-Befehl

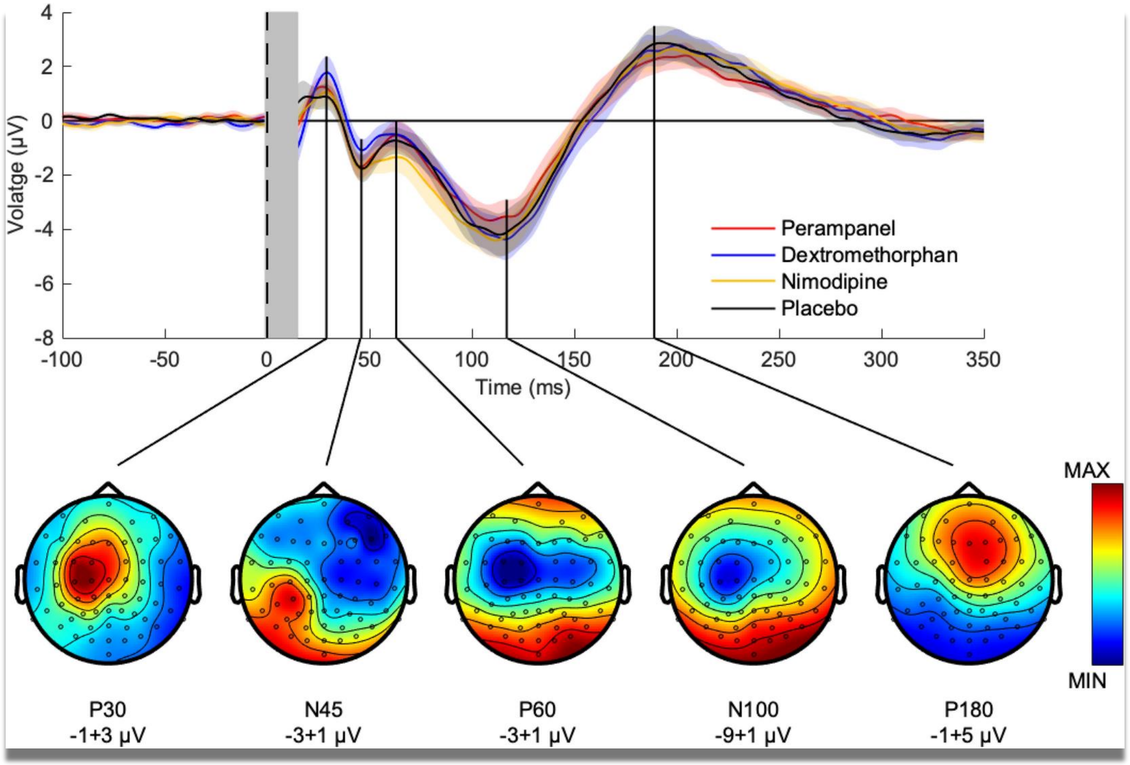
- Topographien
  - plot_topomap



Visualisierung von EEG Daten

- Lineplots
  - Plot-Befehl

- Signalverlauf über die Zeit
  - gemittelt über ausgewählte Kanäle
  - gemittelt über Versuchspersonen

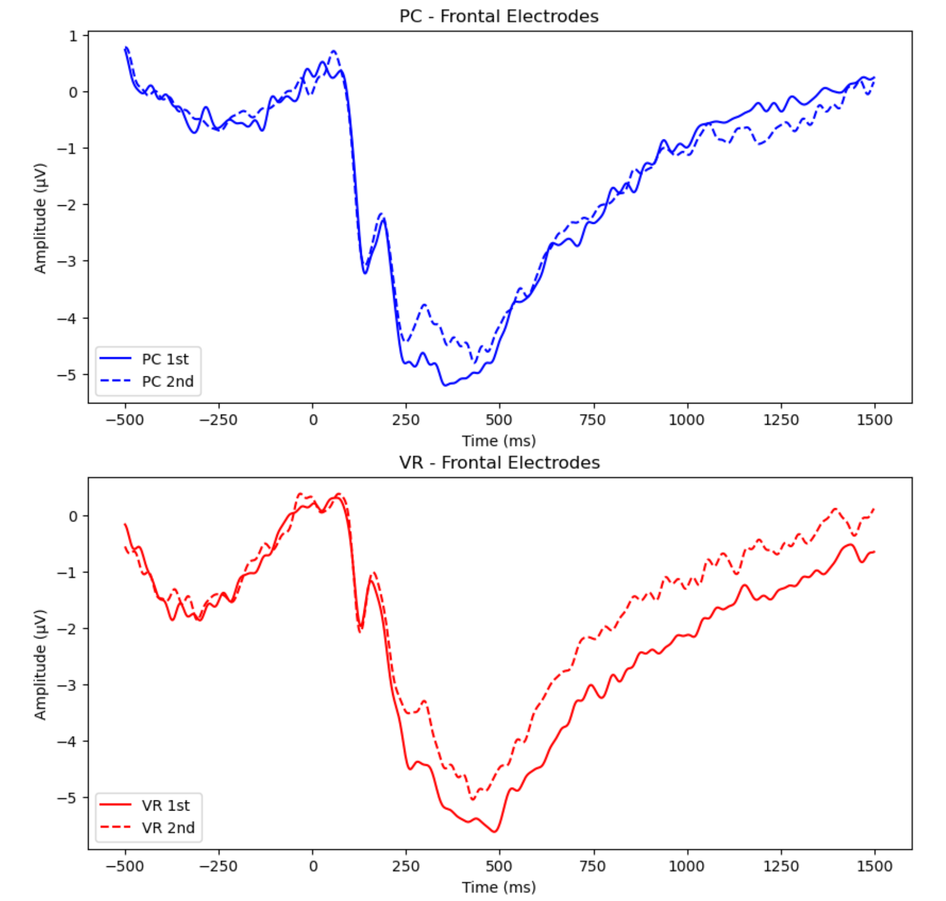


Visualisierung von EEG Daten

A = Bedingungen = 1
B = Elektrodengruppe = 6
Excel = 2d
Panel = pp 11/12/13 ... 16 Elektrodengruppe = 3d mgl. Plot
pp 11/12
11/12

Diffus. =

```python
# Zeitvektor
# Beispiel: -500 ms bis 1500 ms, d.h. 1024 Datenpunkte bei 512 Hz
t = np.linspace(-500, 1499, 1024)

# Elektroden definieren
selected_electrodes = [94, 95, 96, 93, 92, 91, 81, 82, 83, 80, 79, 78, 72, 73, 74]

# Berechne den Mittelwert über die Elektroden und Versuchspersonen
Cond1_mean_elec_vp = np.mean(Cond_1[selected_electrodes, :, :], axis=(0, 2))
Cond2_mean_elec_vp = np.mean(Cond_2[selected_electrodes, :, :], axis=(0, 2))

# Plot Lineplots
plt.plot(t, Cond1_mean_elec_vp, label = 'Condition 1', color = 'blue')
plt.plot(t, Cond2_mean_elec_vp, label = 'Condition 1', color = 'red')
plt.set_xlabel('Time (ms)')
plt.set_ylabel('Amplitude (µV)')
plt.set_title('Lineplot Condition 1')

plt.show()
```

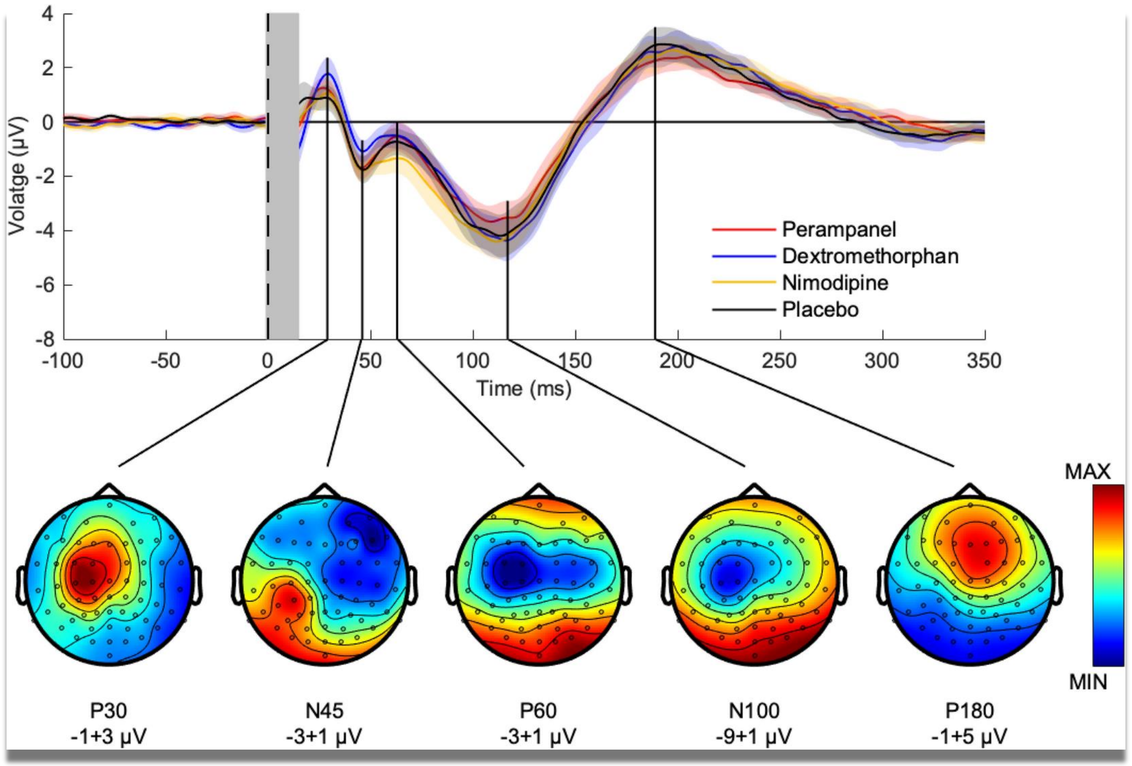


Visualisierung von EEG Daten

- Topographien
  - plot_topomap

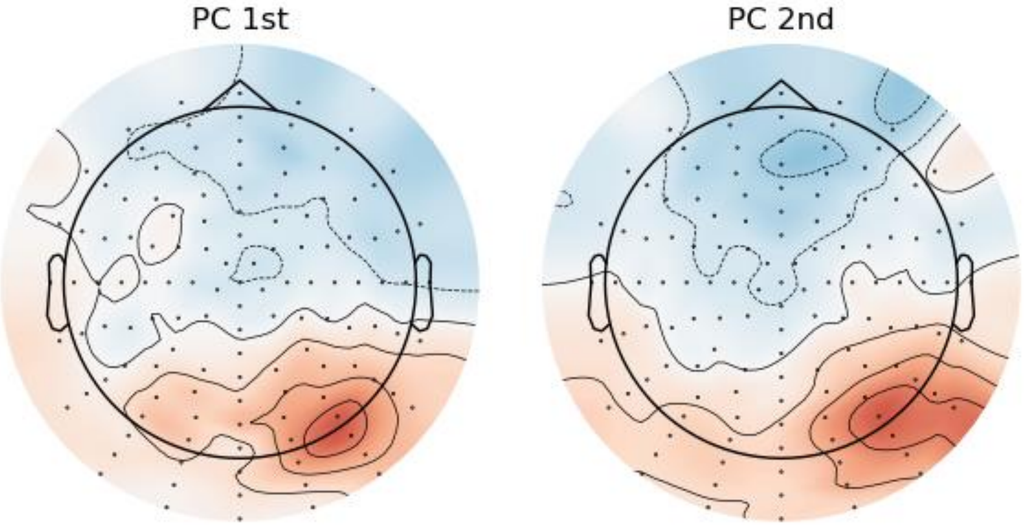
Verlust Zeitinformationen

- <u>Räumliche</u> Signalverteilung
  - gemittelt über ein ausgewähltes Zeitfenster
  - gemittelt über Versuchspersonen



Visualisierung von EEG-Daten

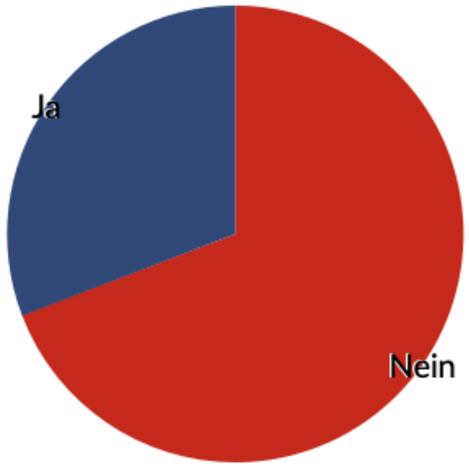
```
# Indizes für das Zeitfenster (z.B. 800 ms bis 1500 ms)
t1_ms = 800
t2_ms = 1500
t1 = np.searchsorted(t, t1_ms)
t2 = np.searchsorted(t, t2_ms)


# Erstelle das Info-Dictionary --> Dummy EEG data
dummy_path = path2load
dummy_EEG = mne.io.read_epochs_eeglab(dummy_path + 'dummyfile128.set')

# Definiere die Amplituden-Range
v_lim = (-3,3)  # Minimal- und Maximalwert für die Farbskala
v_lim_diff = (-2,2)

# Berechne den Mittelwert über das Zeitfenster und alle Versuchspersonen
mean_data_Cond_1 = np.mean(Cond_1[:, t1:t2, :], axis=(1, 2))
mean_data_Cond_2 = np.mean(Cond_2[:, t1:t2, :], axis=(1, 2))


# Topografien
fig, axs = plt.subplots(1, 2, figsize=(9, 6))

mne.viz.plot_topomap(mean_data_Cond_1, dummy_EEG.info, axes=axs[0, 0], show=False, vlim=v_lim)
axs[0, 0].set_title('Condition 1')

mne.viz.plot_topomap(mean_data_Cond_2, dummy_EEG.info, axes=axs[0, 1], show=False, vlim=v_lim)
axs[0, 1].set_title('Condition 2')

plt.tight_layout()
plt.show()
```



Next week!

- **Themenwünsche:**
- Individualisierung von Code in Psychopy

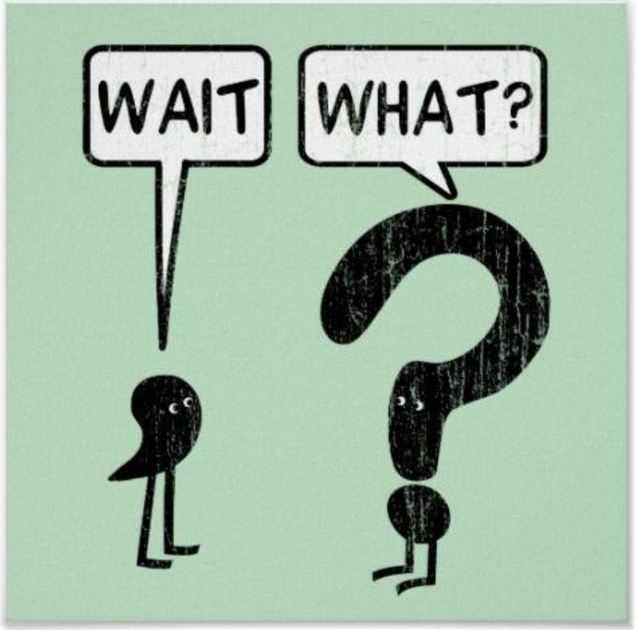
- **Mein(e) Angebot(e):**
- Mini-Exkurs ins Visual Scripting (am Beispiel Unreal Engine)
- Weitere Bearbeitung der (Projekt-)Aufgaben
- Weitere Experimente in PsychoPy
- Allgemeine & Spezifische Fragen zum Programmieren (in Python)



Ja	(31% | 4/13)
Nein	(69% | 9/13)

Your turn!

- Aufgaben: Die auf Stud.IP hochgeladenen EEG-Daten in Python importieren und hübsch visualisieren. Als Hilfestellung liegt ein Blueprint bereit.
  - Lineplots: Verschiedene EKPs gegeneinander plotten.
  - Topos: Verschiedene Bedingungen gegenüberstellen.
  - Tipp: Subplots!
- Optional: Statistiken für den Vergleich der Bedingungen rechnen. Existieren statistisch signifikante Unterschiede?
- Evaluation des Seminars ☺



Evaluation